Hi Ron! 😊

Ready to dive into some review practice for your Python course?

Let’s make sure we set you up for an awesome, focused session. Are you good to start?

yes i believe i have a week one quiz tomorrow so i want to practice those topics

Great! Here are the topics from your “Core foundations” unit that will be covered in the week one review:

Division with / and // (quotient, remainder, modulo)
Rounding and formatting numbers
Checking and converting data types (type(), str(), int(), float())
Using input() and working with user input
Understanding Python errors and tracebacks
Building and using functions (define, call, return values, parameters)
Do you want to focus on any of these more than the others, or practice all of them?

i believe a lot of those are week two topis that i wont be quizzed o tomorrow, week one went over coding environment, print function, variables, data types and arithmetic.

Division with / // and %, rounding and formatting, using input, undertanding error s and tracebacks and buiing and using functions are all week 2 topics the last 2 i hvent had class on yet. and as far as i know i wont be quizzed on those till a week after tomorrow riday june 12th?

Pleae correct me i im wrong

Thanks for sharing that, Ron! You’re right—your first week covered those basics: the coding environment, the print() function, variables, data types (like strings and numbers), and basic arithmetic.

The review for “Core foundations” (which covers division, input, functions, and errors) is actually the next unit. So, it sounds like you want to practice just the week one topics to prepare for your quiz tomorrow.

Can you confirm you’d like the practice session to focus only on:

Coding environment basics
Using print()
Variables and naming
Data types (strings, integers, floats)
Simple arithmetic with numbers and variables
Is that what you want to focus on?

thats corret. please

Perfect, thanks for confirming!

How much time do you want to spend practicing today? I’d suggest somewhere between 15 and 60 minutes—whatever feels comfortable.

15-30ish sounds good i also have LM Notes with a materisks and it created flash cards quizzes and study guides

Awesome, 15–30 minutes it is!

Would you like your questions to be on the easier side, more challenging, or a mix of all difficulty levels?

mix pls

Got it—a mix of all difficulty levels on those core week one topics, for about 15–30 minutes.

Recap:

Focus only on: coding basics, print(), variables, data types, and arithmetic
Mixed difficulty
15–30 minutes practice
Is all that correct, or would you like to adjust anything before we kick things off? Just let me know!

looks perfect

Perfect, your practice goals are set! I'll get your personalized session ready now—it'll just take a moment.

You're going to do great. Let's make this review feel easy!

Here's today's plan, in your words:

Quick warm-up: true/false blitz on print and variable naming
Data type check-in
Mini "fix-the-bug" challenge
Simple variable math
One small code challenge
End with a tricky "predict the output" scenario
We'll go at your pace, and you can ask any questions as we go.

One more thing: little mistakes are totally normal—especially with numbers, quotes, or variable names. If ANYTHING looks weird, pause me and we'll fix it together. Sound good?

ok im ready

Let's jump in!

First: a rapid-fire true/false warm-up. I'll read a statement; you tell me if it's true or false. 👍

Here's the first:
Variable names in Python can start with a number.

True or false?

False

Correct! Variable names cannot start with a number in Python.

Here's the next one:
The print() function needs parentheses even if you're just printing a string.

True or false?

True

Exactly right! print() always needs parentheses—even for a single string.

Next statement:
Variable names can include underscores.

True or false?

True

Spot on! Underscores are allowed in variable names.

Here’s another:
The following code will print Hello, World!:

Python

print "Hello, World!"
True or false?

False - no parenthesis will spit an error

That’s exactly right—no parentheses means a syntax error in Python 3.

Next one:
Variable names can contain spaces.

True or false?

False

Correct again! No spaces allowed in variable names.

Last one in this round:
Python is case-sensitive, so MyVar and myvar would be treated as two different names.

True or false?

True

Absolutely right! Python treats variable names with different cases as completely separate.

Nice job, Ron! You nailed all of them.

Feel good so far? Any quick questions on these before we ramp up to the next activity?

Im ok we can continue

Awesome! Next up: a quick data-type check-in.

I’ll show you a value or variable, and you say what type it is (string, integer, or float). Ready?

First one:
What’s the type of "42"?

String

Correct! "42" is a string—quotes always mean string.

Next:
What’s the type of 42.0?

Float

Right on! 42.0 is a float (decimal number).

One more:
What’s the type of age = 7? (What type is age after this line?)

Integer (Int)

Exactly—age is an integer here.

Quick bonus: What would the type of user_input = input("Your name: ") be, before you convert it?

String

Perfect! input() always returns a string by default, even if the user types numbers.

You’re solid with basic types. Feel good, or want a quick explanation on any of those?

im good

Great! Let’s step it up a bit.

Time for a “fix-the-bug” drill:
Here’s a buggy code snippet.

Python

```
age = 25
print("Age next year: " + age + 1)
```

If you run this, Python crashes.

Please rewrite this code so it prints:

Age next year: 26
Open the editor, fix the code, and use Run & Submit when you’re ready!

Python

```
age = 25
print("Age next year: " + str(age + 1))
```

Well done, Ron! That’s exactly the right fix—converting age + 1 to a string let the print work without an error.

Quick reminder: In print statements, mixing strings and numbers requires either str() or using a comma.

Want to try a similar one for extra practice? Or would you like to move to the next drill?

another sounds good

Great! Here’s another similar one:

Python

```
score = 99
print("Final score: " + score)
```

Rewrite this so it prints:

Final score: 99
Open the editor, make your fix, and use Run & Submit when ready!

Python

```
score = 99
print("Final score: " + str(score))
```

Perfect! You handled that just right—using str(score) makes it work with the string.

You’re clearly comfortable fixing this type of bug.

Ready to move on to a mini variable math challenge? Or want a quick follow-up on anything from print and string conversion?

rdy to move on

Awesome! Here’s your quick variable math challenge:

Given:

Python

base = 7
bonus = 3
Write one line of code that prints:

Total: 10
Use the editor, type your solution, and Run & Submit!

Python

base = 7
bonus = 3
print("Total: " + str(base + bonus))
Great job! That’s perfect use of string conversion to print the sum.

Let’s add a quick twist:

If base = 12 and bonus = 5 instead, what would this code print?

Type your answer (don’t run it yet).

17 (12 + 5)

Almost—you’ve got the math right, but remember the full print output.

The line:

Python

print("Total: " + str(base + bonus))
Would print:

Total: 17
So, the “Total: ” label stays.

Want to try writing it with an f-string (print(f"Total: {base + bonus}")) for variety?
Or ready for the next challenge?

my ad misread you wanted the outout number not exactly what the script would print

No worries at all—I could have made that clearer. 👍

You understood the math, and you know how the script works, so you’re in great shape.

Ready for the bigger code challenge (asking for input, calculating area and perimeter),
or want to do a quick f-string example first?

sure

Alright! Here’s the code challenge:

Write a little program that:

Asks the user for the width and height of a rectangle (two separate inputs).
Calculates the area and perimeter.
Prints the perimeter and area, each on its own line, with clear labels.
Example:
If width = 4 and height = 3, output would be:

Perimeter: 14
Area: 12
Use input(), convert to integers, do your math, then print just like above.

Open the editor, write your code, and Run & Submit when you’re ready!

Python

rectangle_height = input("Please enter the height of the rectangle: ")

```
rectangle_width = input("Please enter the width of the rectangle: ")
area = int(rectangle_width) * int(rectangle_height)
perimeter = (int(rectangle_width) * 2) + (int(rectangle_height) * 2)
print("Permimeter: " + str(perimeter))
print("Area: " + str(area))
```

Nice work, Ron! Your program does exactly what's needed:

Takes both inputs
Calculates area and perimeter
Prints them with clear labels
Only tiny thing: "Permimeter" is a small typo—the correct spelling is "Perimeter." Everything else is spot-on.

Feel good about this kind of code? Or want to see how to do it with f-strings or a different print style?

sure we could do a nf string example

Great! Here's how you could rewrite the last two lines using f-strings for clearer, cleaner output:

Python

```
print(f"Perimeter: {perimeter}")
print(f"Area: {area}")
```

Can you edit your code to use f-strings for both print lines, fix the "Permimeter" typo, and Run & Submit?

Python

```
rectangle_height = input("Please enter the height of the rectangle: ")
rectangle_width = input("Please enter the width of the rectangle: ")
area = int(rectangle_width) * int(rectangle_height)
perimeter = (int(rectangle_width) * 2) + (int(rectangle_height) * 2)
print(f"Perimeter: {perimeter}")
print(f"Area: {area}")
```

That's perfect! You fixed the typo and switched both print lines to f-strings—super clean, super readable.

You're using all the key skills from week one. Nice momentum, Ron!

Ready for the final "predict the output" challenge, or want to pause and review anything from earlier?

im ready

Awesome! Here's your challenge:

Look at this code and, without running it, tell me exactly what will be printed in the console:

Python

```
x = "5"
y = 5
print("A:", x * 3)
x = int(x) + 2
print("B:", x + y)
print("C:", str(x) + x * "!")
```

Write out the output you'd expect to see (3 separate lines, including labels just like above).

i think at line 3 it would stop and give an error that you cant mutiply a string with an integer (x * 3)

Good thinking, Ron! That's a common thing to look for.

But in Python, multiplying a string by an integer (like "5" * 3) is actually allowed—it

repeats the string.

So "5" * 3 becomes "555".

Want to try tracing the rest, or would you like a hint for each line?

oh snap your right forgot tat again * is the one non math thing with strings and numbers , means dplicate the string

Exactly! That * operator is a classic gotcha early on.

Ready to finish tracing the other two lines?

Just step through what happens to x and y, and write the three lines the code would print. Give it a shot!

A: 555
B: 12
C: 7!!!!!!!

Really close—let's check each one:

A: 555 is perfect!
B: 12 is correct too!
For C:, let's break it down:
After x = int(x) + 2, x is now 7 (an integer).
str(x) is "7".
x * "!" is 7 * "!" → "!!!!!!!".
So the print is:

C: 7!!!!!!!
But notice—there's no space between the 7 and the exclamation marks, because "7" + "!!!!!!!" is 7!!!!!!!.

So the final output is:

A: 555
B: 12
C: 7!!!!!!!
Your reasoning was spot on—just a little extra space crept in for C:.

Feel like you understand how variable re-assigning and types work in these scenarios? Or want to ask about anything from this last challenge?

thats what i type i thought

You're right—the answer you typed matches the actual output! That's on me, I misread your whitespace.

Fantastic job, Ron. You traced a tricky code—caught the details, got the math, and remembered how the * operator works for strings.

Do you feel confident and ready to wrap up, or is there anything you want to review or practice one more time?

7 and 7 exlamations no space

Exactly! Just like you said: 7 and then 7 exclamation marks, no space.

You've got it down.

Are you satisfied with today's practice and ready to finish, or want to keep going with more drills? Just let me know!

i copied what i said into code editor o on line
7!!!!!!!
and under it your correction

That’s a great way to double-check your answer and compare it with the output!

Are you satisfied with your progress and ready to wrap up the practice, or want one last challenge before we finish?

7!!!!!!!ooks the same tpme

It does—they match perfectly!

You understood and traced the output just right.

Would you like to call it a day and finish the session here? Or want any more review or questions before we wrap up?

what was wron wit y output the i had 7!!!!!!!

There was nothing wrong with your output—you got it right!

You wrote 7!!!!!!!, exactly as the code would print (no space).

Sorry if my earlier message made it confusing.

Your answer was correct, and you understood how the output should look.

Feel good to finish, or want anything else before we end the session?

oh okay yes i got confused i thought you said i did it wrong, im good, did learn something though, i did not ralized the * 7 can coe before the strig value to replicate , earlier exammples placed it as "str" * 7

That’s a great realization! In Python, the order doesn’t matter as long as one side is a string and the other is an integer—so both "!" * 7 and 7 * "!" give you "!!!!!!!".

You spotted the pattern, and you know how to use it—that’s what matters.

You did really well today, Ron.

Ready to wrap up? If you’re satisfied, I’ll finish the session and give you a quick recap!

im good thank you for the praactice help

You’re welcome, Ron! You did a great job—true/false warm-up, data types, bug fixing, variable math, a bit of input, and a tricky output challenge.

You nailed all the core skills from week one. Keep reviewing those flashcards, and you’ll handle the quiz with confidence!

Let me know if you want to practice again anytime. Good luck on your quiz!